

列表 循位置访问

从静态到动态

❖ 根据是否修改数据结构，所有操作大致分为两类方式

- 静态： 仅读取，数据结构的内容及组成一般不变：get、search
- 动态： 需写入，数据结构的局部或整体将改变：put、insert、remove

❖ 与操作方式相对应地，数据元素的存储与组织方式也分为两种

- 静态： 数据空间整体创建或销毁

数据元素的物理存储次序与其逻辑次序严格一致；可支持高效的静态操作

比如向量，元素的物理地址与其逻辑次序线性对应

- 动态： 为各数据元素动态地分配和回收的物理空间

相邻元素记录彼此的物理地址，在逻辑上形成一个整体；可支持高效的动态操作

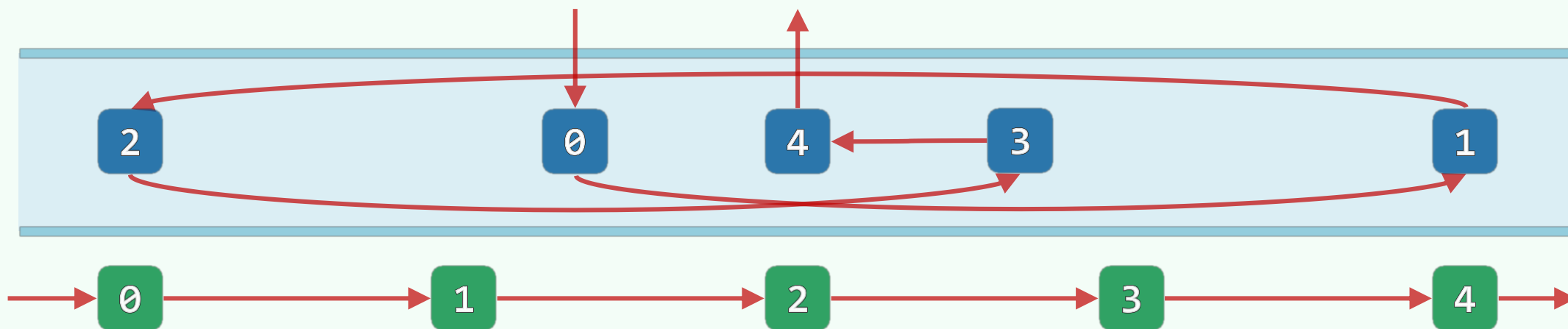
从向量到列表

❖ 列表 (list) 是采用动态储存策略的典型结构

- 其中的元素称作节点 (node) , 通过指针或引用彼此联接
- 在**逻辑**上构成一个线性序列: $L = \{ a_0, a_1, \dots, a_{n-1} \}$

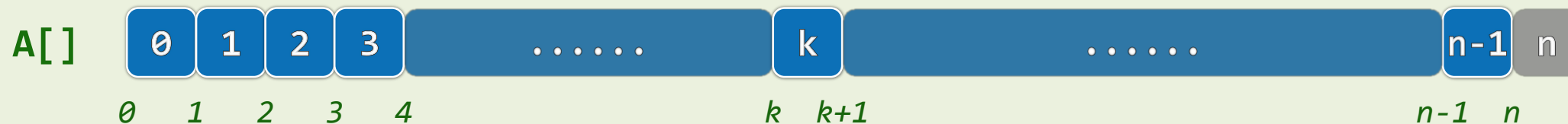
❖ 相邻节点彼此互称前驱 (predecessor) 或后继 (successor)

- 没有前驱/后继的节点称作**首** (first/front) /**末** (last/rear) 节点



从秩到位置

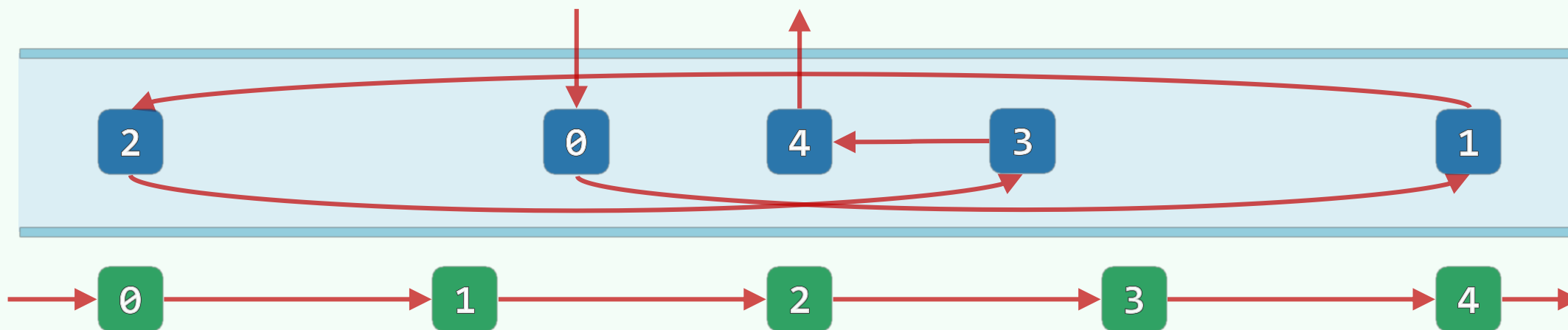
❖ 向量支持循秩访问 (call-by-rank) : 根据元素的秩, 可在 $O(1)$ 时间内直接确定其物理地址



❖ 这种高效的方式, 可否被列表沿用?

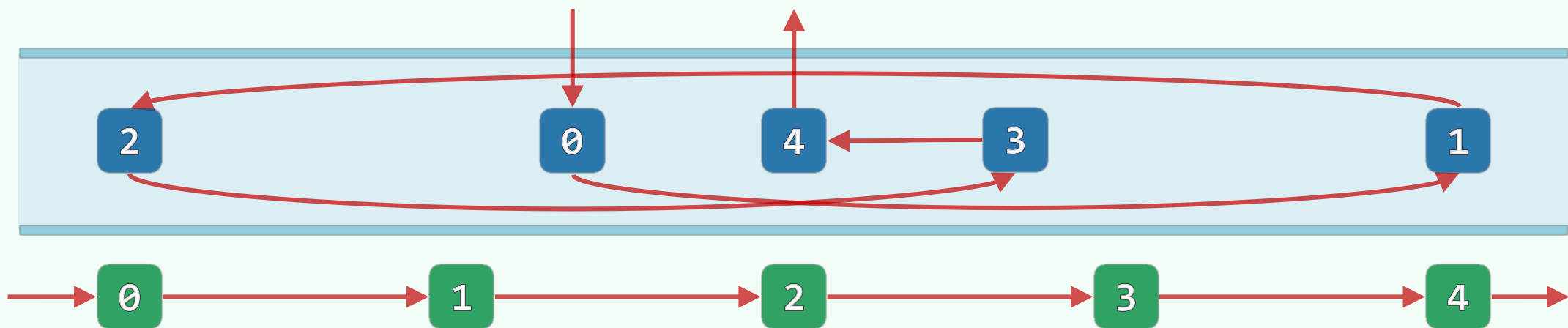
❖ 比如, 从头/尾端出发, 沿后继/前驱引用...

`//List::operator[](Rank r)`



从秩到位置

- ❖ 然而，此时的循秩访问成本过高，已不合时宜
- ❖ 因此，应改用循**位置**访问 (call-by-position) 的方式
 - 亦即，转而利用节点之间的相互**引用**，找到特定的节点
- ❖ 比喻：找到我的**朋友A**的**亲戚B**的**同事C**的**战友D**的...的**同学Z**



列表 接口与实现

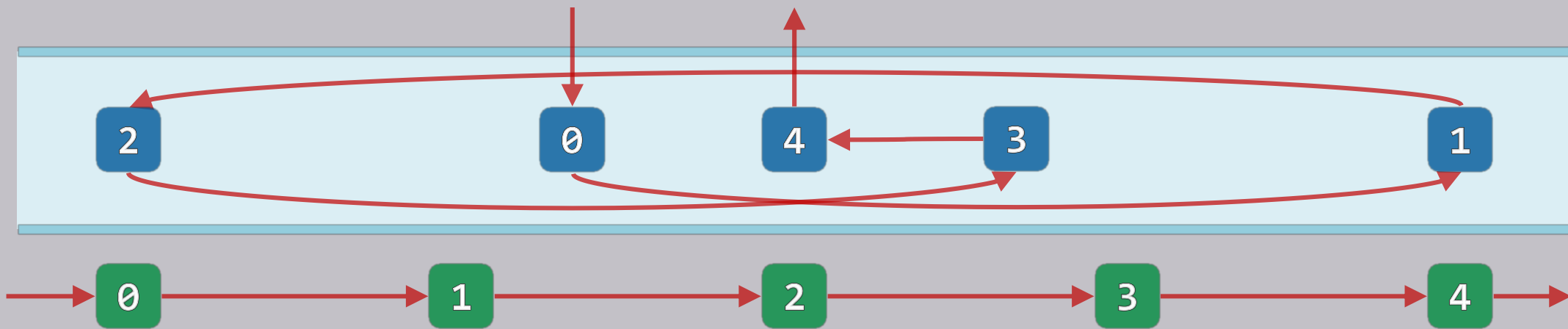
列表节点：ADT接口

❖ 作为列表的基本元素

列表节点首先需要

独立地“封装”实现

操作接口	功能
pred()	当前节点前驱节点的位置
succ()	当前节点后继节点的位置
data()	当前节点所存数据对象
<u>insertAsPred(e)</u>	插入前驱节点，存入被引用对象e，返回新节点位置
<u>insertAsSucc(e)</u>	插入后继节点，存入被引用对象e，返回新节点位置



列表节点：模板类

❖ `template <typename T> using ListNodePosi = ListNode<T>*; //列表节点位置 (C++ .0x)`

❖ `template <typename T> struct ListNode { //简洁起见，完全开放而不再严格封装`

`T data; //数值`

`ListNodePosi<T> pred; //前驱`

`ListNodePosi<T> succ; //后继`

`ListNode() {} //针对header和trailer的构造`

`ListNode(T e, ListNodePosi<T> p = NULL, ListNodePosi<T> s = NULL)`

`: data(e), pred(p), succ(s) {} //默认构造器`

`ListNodePosi<T> insertAsPred( T const & e ); //前插入`

`ListNodePosi<T> insertAsSucc( T const & e ); //后插入`

`};`



列表：ADT接口

操作接口	功能	适用对象
size()	报告列表当前的规模（节点总数）	列表
first(), last()	返回首、末节点的位置	列表
insertAsFirst(e), insertAsLast(e)	将e当作首、末节点插入	列表
insert(p, e), insert(e, p)	将e当作节点p的直接后继、前驱插入	列表
remove(p)	删除位置p处的节点，返回其中数据项	列表
disordered()	判断所有节点是否已按非降序排列	列表
sort()	调整各节点的位置，使之按非降序排列	列表
find(e)	查找目标元素e，失败时返回NULL	列表
search(e)	查找e，返回不大于e且秩最大的节点	有序列表
deduplicate(), uniquify()	剔除重复节点	列表/有序列表
traverse()	遍历列表	列表

列表：模板类

```
#include "listNode.h" //引入列表节点类
```

```
template <typename T> class List { //列表模板类
```

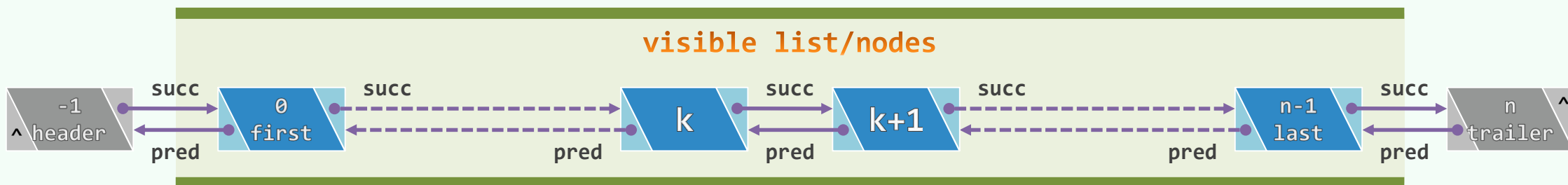
```
private:    int _size; ListNodePosi<T> header; ListNodePosi<T> trailer; //哨兵
```

```
           //头、首、末、尾节点的秩，可分别理解为-1、0、n-1、n
```

```
protected: /* ... 内部函数 */
```

```
public:     /* ... 构造函数、析构函数、只读接口、可写接口、遍历接口 */
```

```
};
```



构造

❖ `template <typename T> void List<T>::init() { //初始化, 创建列表对象时统一调用`

`header = new ListNode<T>;`

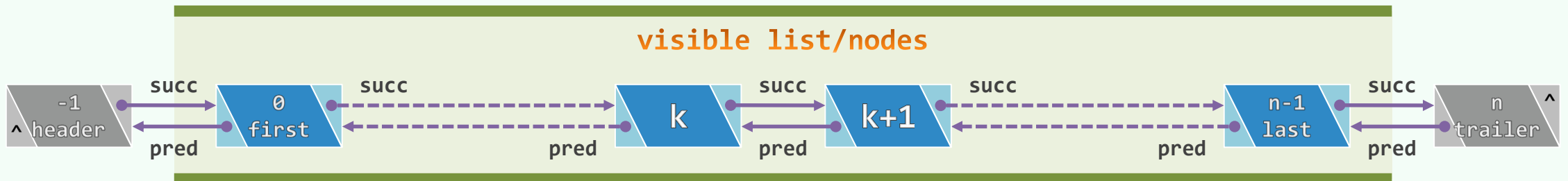
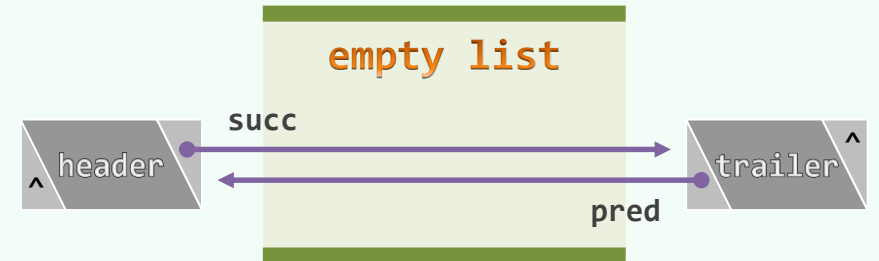
`trailer = new ListNode<T>;`

`header->succ = trailer; header->pred = NULL;`

`trailer->pred = header; trailer->succ = NULL;`

`_size = 0;`

`}`



循秩访问

❖ 通过重载下标操作符，可**模仿**向量的循秩访问方式

❖ `template <typename T> //assert:  $0 \leq r < \text{size}$`

`T List<T>::operator[] ( Rank r ) const { //  $O(r)$ 效率低下，可偶尔为之，却不宜常用`

`ListNodePosi<T> p = first(); //从首节点出发`

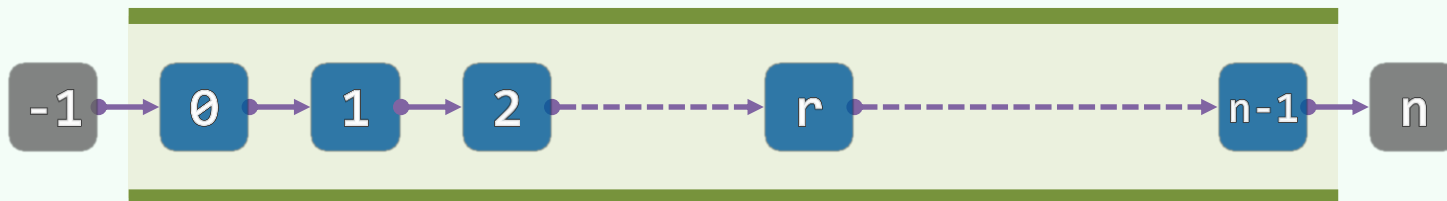
`while (  $0 < r--$  ) p = p->succ; //顺数第 $r$ 个节点即是`

`return p->data; //目标节点`

`} //秩 == 前驱的总数`

❖ 时间复杂度为 $O(r)$

均匀分布时，期望复杂度为 $(1 + 2 + 3 + \cdots + n)/n = O(n)$

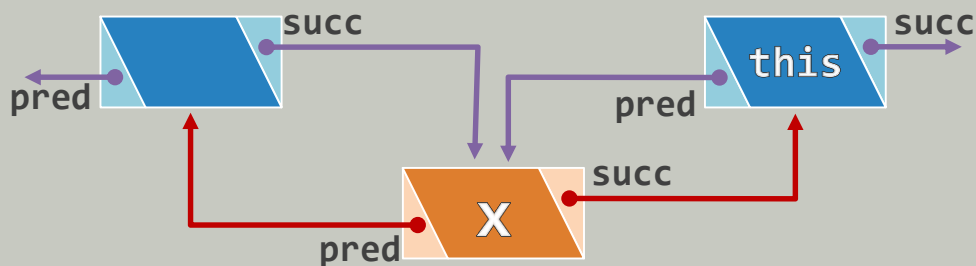
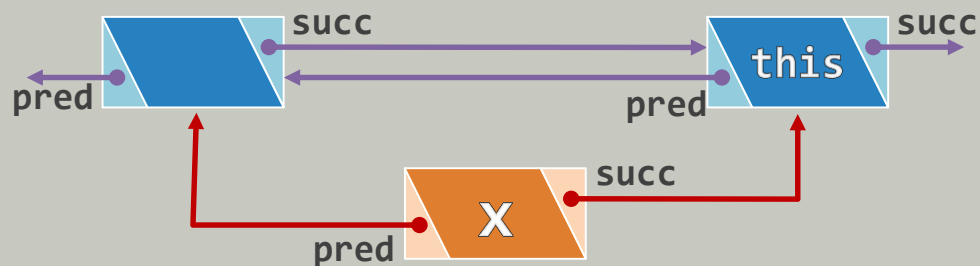


列表 插入与删除

插入：思路 + 过程

❖ `template <typename T> ListNodePosi<T> List<T>:: //e当作p的前驱插入`

`insert(T const & e, ListNodePosi<T> p) { _size++; return p->insertAsPred( e ); }`



插入：实现

❖ template <typename T> //前插入算法（后插入算法完全对称）

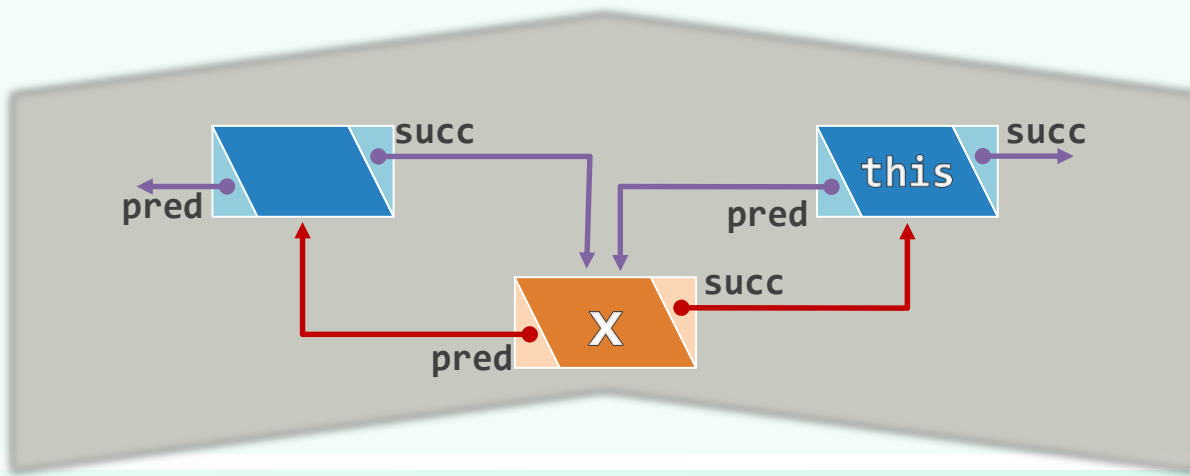
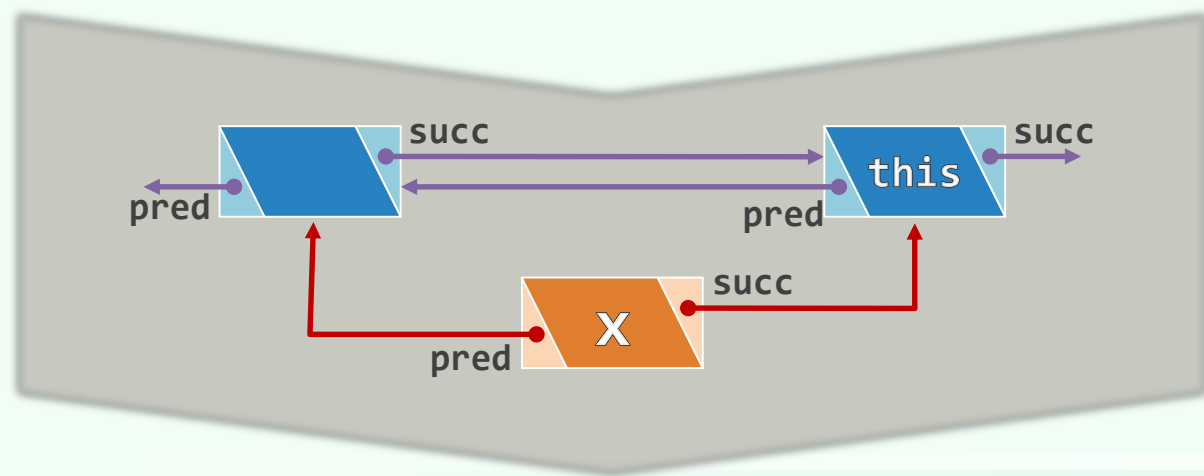
```
ListNodePosi<T> ListNode<T>::insertAsPred( T const & e ) { //O(1)
```

```
    ListNodePosi<T> x = new ListNode( e, pred, this ); //创建（耗时100倍）
```

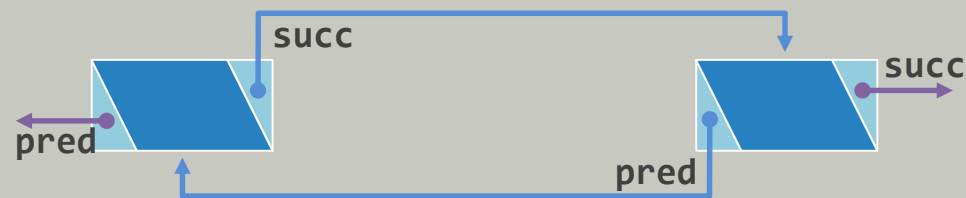
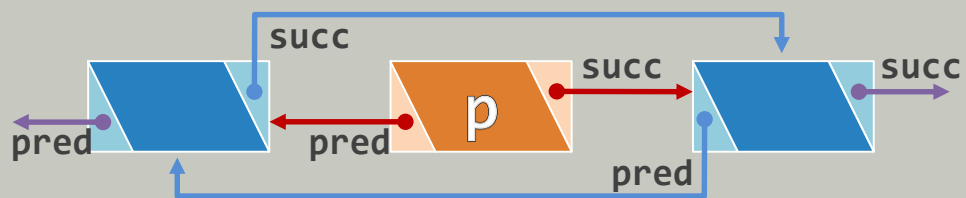
```
    pred->succ = x; pred = x; //次序不可颠倒
```

```
    return x; //建立链接，返回新节点的位置
```

```
} //得益于哨兵，即便this为首节点亦不必特殊处理——此时等效于insertAsFirst(e)
```



删除：思路 + 过程



删除：实现

❖ `template <typename T> //删除合法位置p处节点，返回其数值`

```
T List<T>::remove( ListNodePosi<T> p ) { //O(1)
```

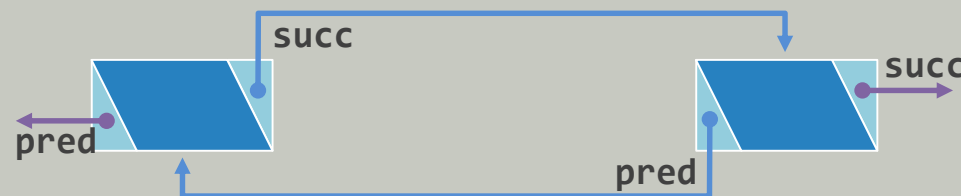
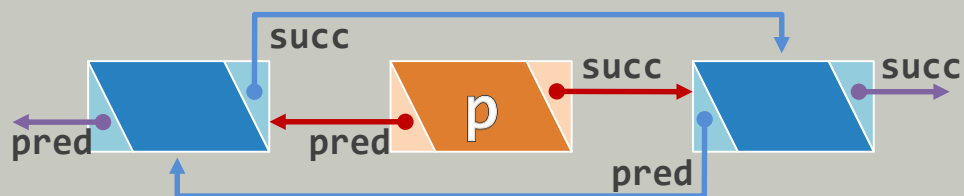
```
    T e = p->data; //备份待删除节点数值（设类型T可直接赋值）
```

```
    p->pred->succ = p->succ;
```

```
    p->succ->pred = p->pred;
```

```
    delete p; _size--; return e; //返回备份数值
```

```
}
```



列表

无序列表：构造与析构

copyNodes() + 构造

```
template <typename T> void List<T>::copyNodes( ListNodePosi<T> p, int n ) { //O(n)
```

```
    init(); //创建头、尾哨兵节点并做初始化
```

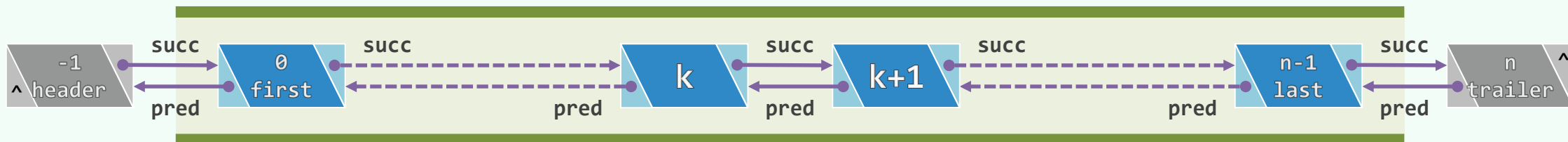
```
    while ( n-- ) { //将起自p的n项依次作为末节点
```

```
        insertAsLast( p->data ); //插入
```

```
        p = p->succ;
```

```
    }
```

```
}
```



```
List<T>::List( List<T> const & L ) { copyNodes( L.first(), L._size ); }
```

clear() + 析构

❖ `template <typename T> List<T>::~~List() //列表析构`
`{ clear(); delete header; delete trailer; }` //清空列表, 释放头、尾哨兵节点

❖ `template <typename T> int List<T>::clear() { //清空列表`

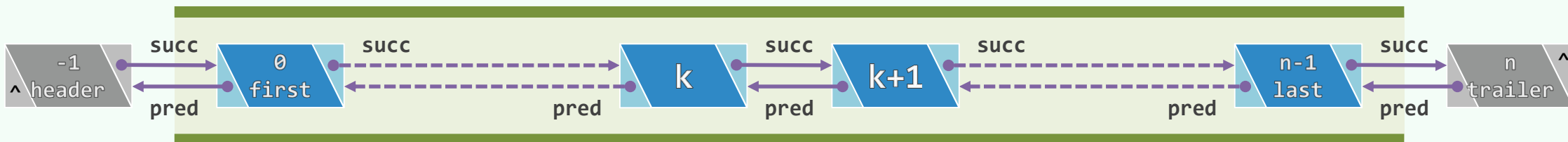
`int oldSize = _size;`

`while ( 0 < _size ) //反复删除首节点,  $O(n)$`

`remove( header->succ );`

`return oldSize;`

`}`



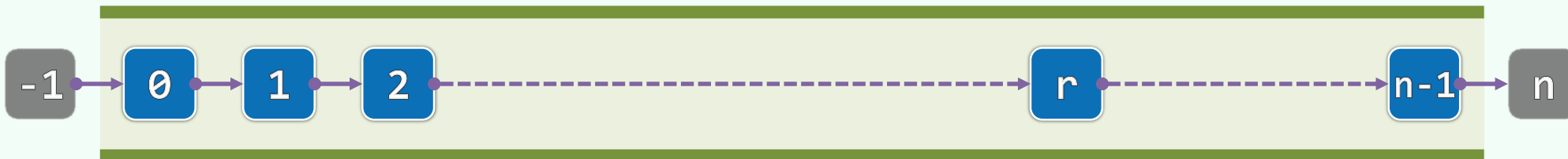
❖ 若`remove( header->succ )`改作`remove( trailer->pred )`呢?

列表

无序列表：查找与去重

查找

```
template <typename T> //0 ≤ n ≤ rank(p) < _size  
ListNodePosi<T> List<T>::find( T const & e, int n, ListNodePosi<T> p ) const {  
    while ( 0 < n-- ) //自后向前逐个比对  
        if ( e == ( p = p->pred ) ->data ) //假定类型T已重载 “==”  
            return p; //在p的n个前驱中，等于e的最靠后者  
    return NULL; //失败  
} //O(n)
```



```
ListNodePosi<T> find( T const & e ) const { return find( e, _size, trailer ); }
```

去重

```
template <typename T> int List<T>::deduplicate() {
```

为什么不是删除p?

```
    int oldSize = _size;
```

```
    ListNodePosi<T> p = first();
```



```
    for ( Rank r = 0; p != trailer; p = p->succ ) //O(n)
```

```
        if ( ListNodePosi<T> q = find( p->data, r, p ) ) //O(n)
```

```
            remove ( q );
```



```
        else
```

```
            r++; //无重前缀的长度
```



```
    return oldSize - _size; //删除元素总数
```

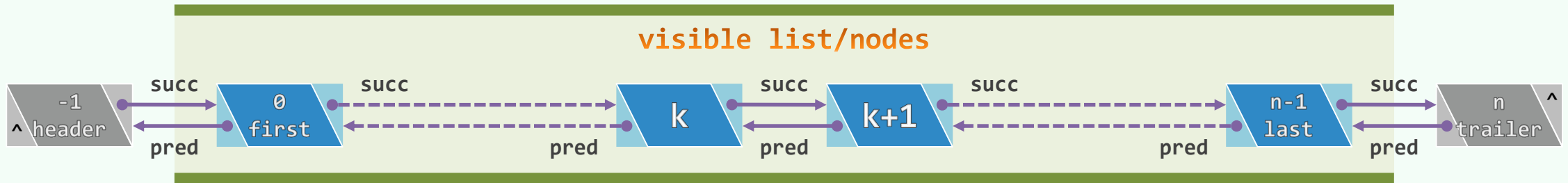
```
} //正确性及效率分析的方法与结论, 与Vector::deduplicate()相同
```

列表

无序列表： 遍历

traverse(): 函数指针/函数对象

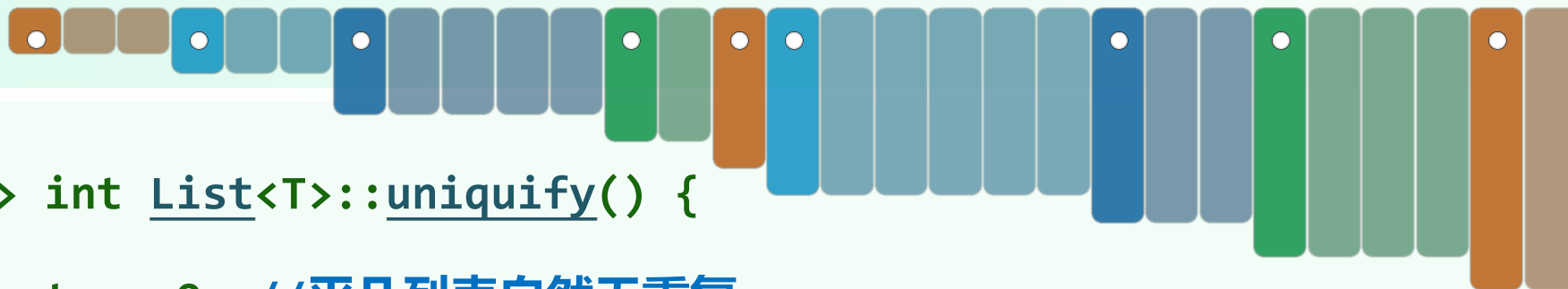
```
template <typename T> void List<T>::traverse( void ( * visit )( T & ) )  
  
{ for( NodePosi<T> p = header->succ; p != trailer; p = p->succ ) visit( p->data ); }
```



```
template <typename T> template <typename VST> void List<T>::traverse( VST & visit )  
  
{ for( NodePosi<T> p = header->succ; p != trailer; p = p->succ ) visit( p->data ); }
```

列表
有序列表： 唯一化

uniquify()



```
❖ template <typename T> int List<T>::uniquify() {  
    if ( _size < 2 ) return 0; //平凡列表自然无重复  
    int oldSize = _size; //记录原规模  
    ListNodePosi<T> p = first(); ListNodePosi<T> q; //各区段起点及其直接后继  
    while ( trailer != ( q = p->succ ) ) //反复考查紧邻的节点对(p,q)  
        if ( p->data != q->data ) p = q; //若互异，则转向下一对  
        else remove(q); //否则（雷同），删除后者  
    return oldSize - _size; //规模变化量，即被删除元素总数  
} //只需遍历整个列表一趟， $O(n)$ 
```

列表

有序列表： 查找

search()

❖ 在有序列表内节点p的n个（真）前驱中，找到不大于e的**最靠后者**

❖ `template <typename T> // assert: 0 <= n <= rank(p) < _size`

```
ListNodePosi<T> List<T>::search( T const & e, int n, ListNodePosi<T> p ) const {
```

```
    do { p = p->pred; n--; } //从右向左
```

```
    while ( ( -1 < n ) && ( e < p->data ) ); //逐个比较，直至命中或越界
```

```
    return p;
```

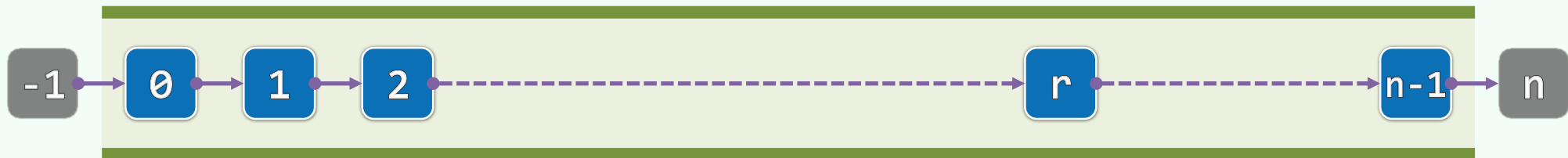
```
}
```



❖ 失败时，返回区间左边界的前驱（可能是header）

性能 + 拓展

- ❖ 最好 $O(1)$ ，最坏 $O(n)$ ；等概率时平均 $O(n)$ ，正比于区间宽度



- ❖ 语义与向量相似，便于插入排序等后续操作： `insert( search( e, r, p ), e )`
- ❖ 为何未能借助有序性提高查找效率？实现不当，还是根本不可能？
- ❖ 按照循位置访问的方式，**物理**存储地址与其**逻辑**次序无关

依据秩的**随机访问**无法高效实现，而只能依据元素间的引用**顺序访问**

列表 选择排序

起泡排序：温故知新

❖ 每趟扫描交换都需 $O(n)$ 次比较、 $O(n)$ 次交换；然而其中， $O(n)$ 次交换完全没有必要

❖ 扫描交换的实质效果无非是

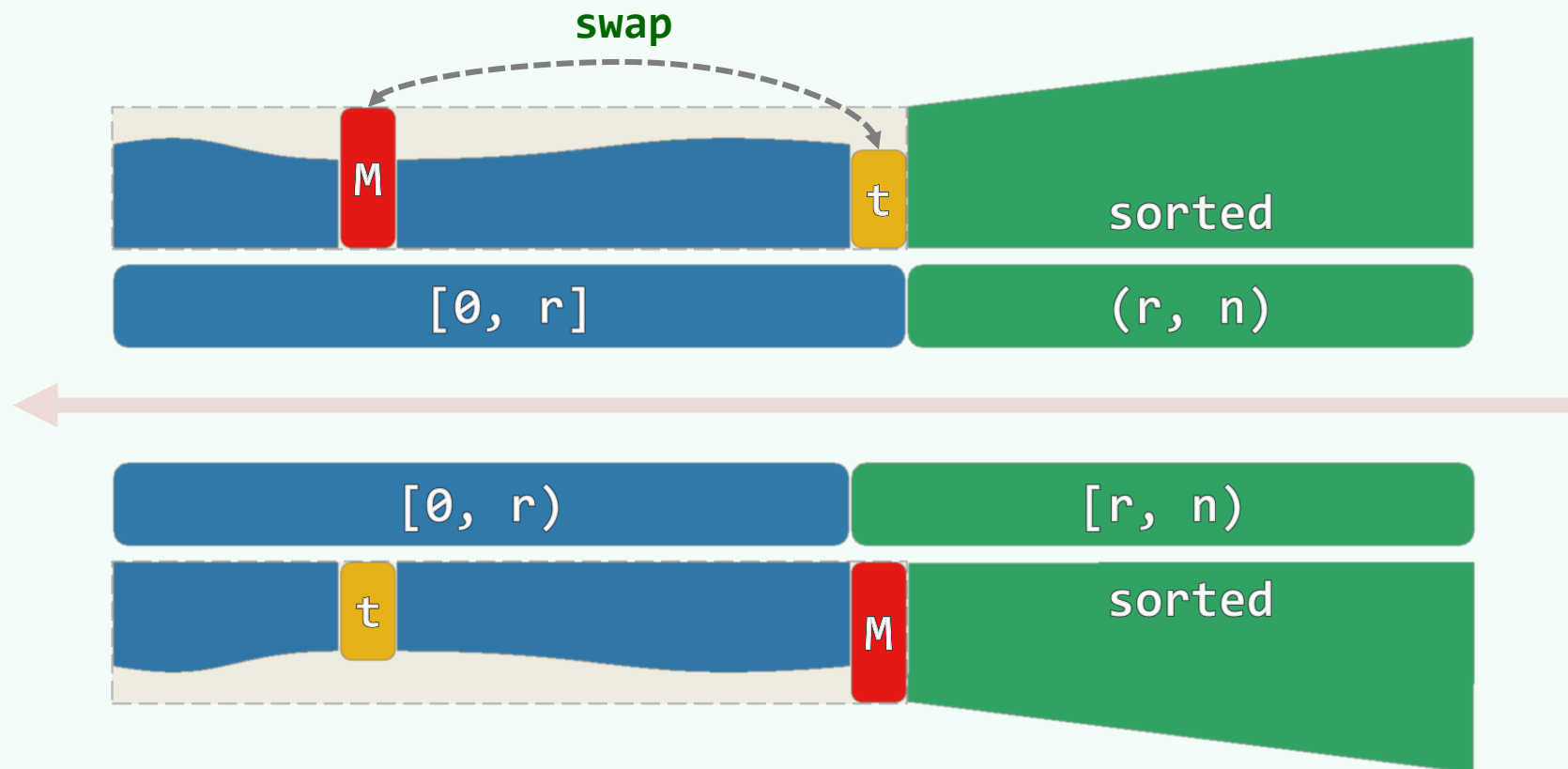
- 通过比较找到当前的

最大元素 M ，并

- 通过交换使之就位

❖ 如此看来

在经 $O(n)$ 次比较确定 M 之后，仅需一次交换即足矣



selectionSort()

//对列表中起始于位置p、宽度为n的区间做选择排序, valid(p) && rank(p) + n <= size

```
template <typename T> void List<T>::selectionSort( ListNodePosi<T> p, int n ) {
```

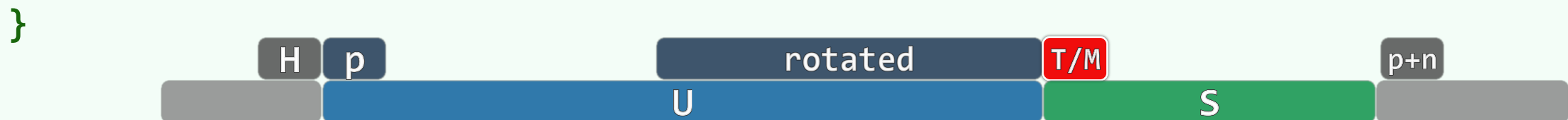
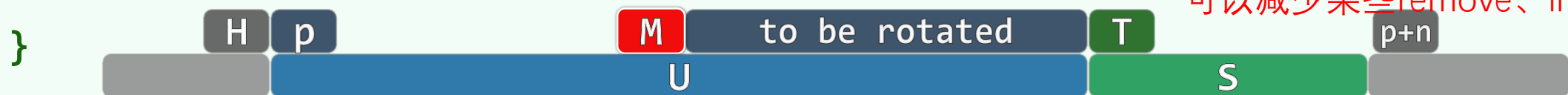
```
    ListNodePosi<T> head = p->pred, tail = p;
```

```
    for ( int i = 0; i < n; i++ ) tail = tail->succ; //待排序区间为(head, tail)
```

```
    while ( 1 < n ) { //反复从 (非平凡) 待排序区间内找出最大者, 并移至有序区间前端
```

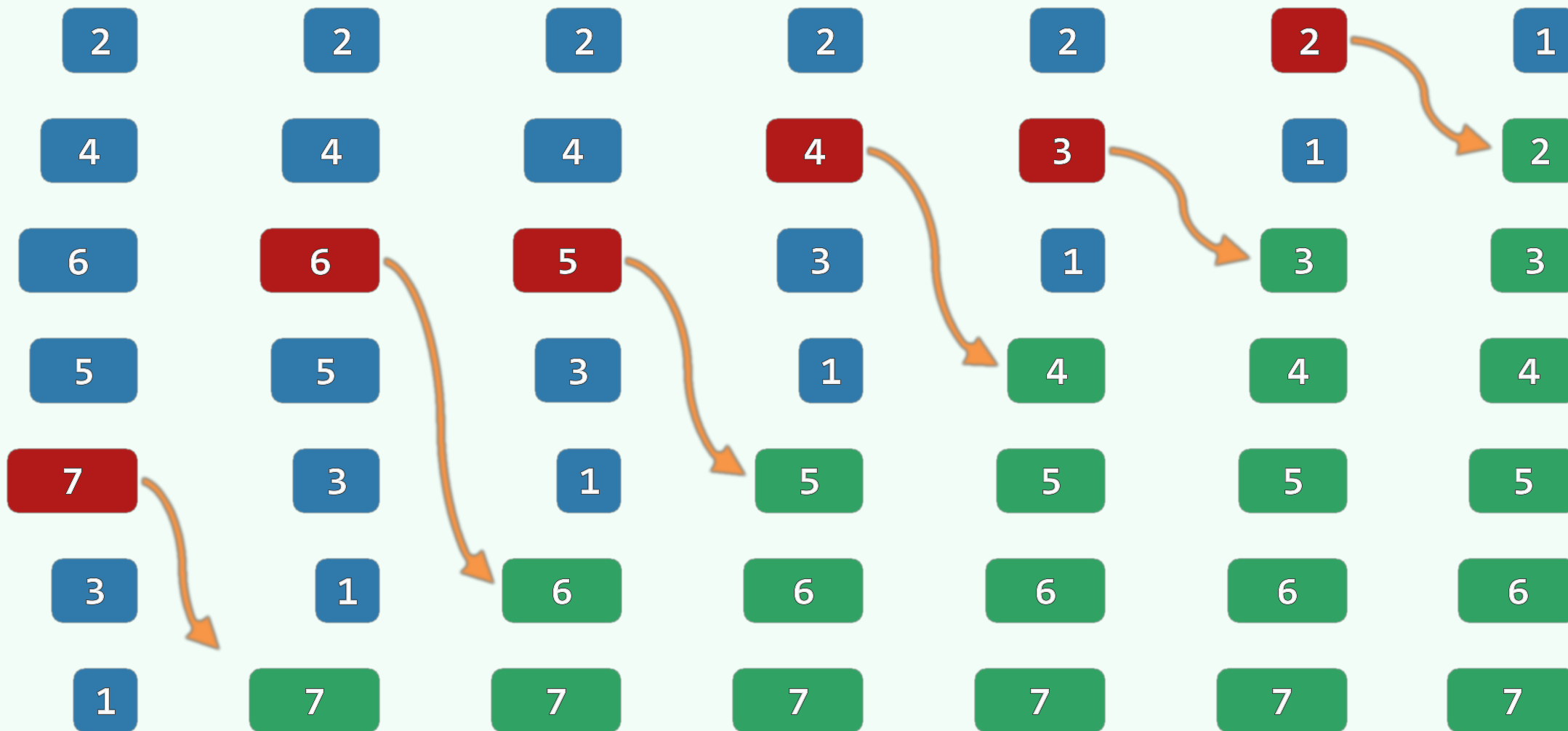
```
        insert( remove( selectMax( head->succ, n ) ), tail ); //可能就在原地...
```

```
        tail = tail->pred; n--; //待排序区间、有序区间的范围, 均同步更新
```



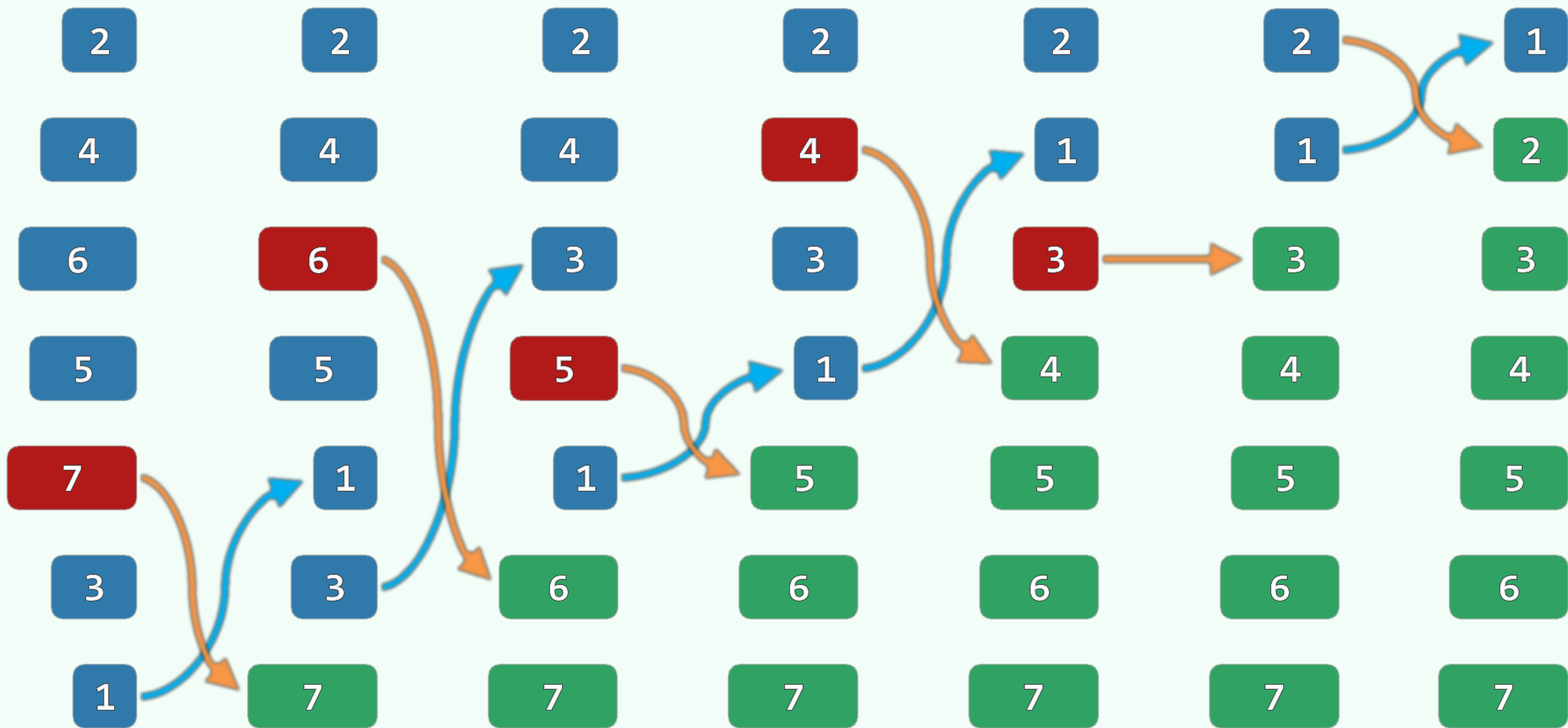
平移法

适合向量、列表吗？



交换法

适合向量、列表吗？



selectMax()

❖ template <typename T> //从起始于位置p的n个元素中选出最大者, $1 < n$

```
ListNodePosi<T> List<T>::selectMax( ListNodePosi<T> p, int n ) { //Θ(n)
```

```
    ListNodePosi<T> max = p; //最大者暂定为p
```

```
    for ( ListNodePosi<T> cur = p; 1 < n; n-- ) //后续节点逐一与max比较
```

```
        if ( ! lt( (cur = cur->succ)->data, max->data ) ) //data ≥ max
```

```
            max = cur; //则更新最大元素位置记录
```

==必要吗?

```
    return max; //返回最大节点位置
```

```
}
```



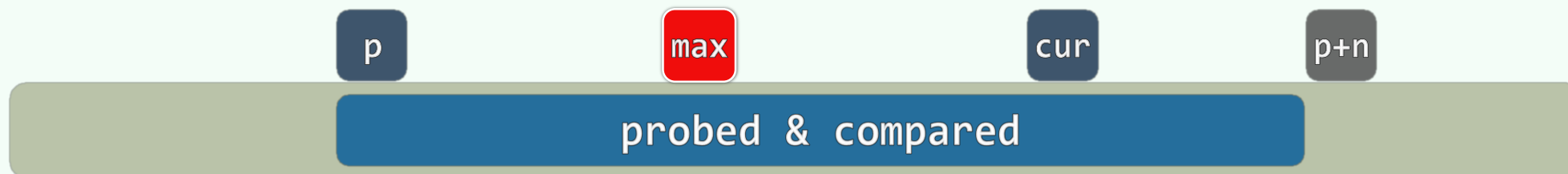
稳定性

- ❖ 有多个重复元素同时命中时，往往需要按照某种**附加的**约定，返回其中**特定的某一个**
- ❖ 比如，通常都约定 “**靠后者**优先返回”
- ❖ 为此，必须采用比较器 `lt()` 或 `ge()`，即等效于后者优先

2	6a	4	6b	3	0	<u>6c</u>	1	5	7	8	9
2	6a	4	6b	3	0	1	5	6c	7	8	9
2	6a	4	3	0	1	5	6b	6c	7	8	9
2	4	3	0	1	5	6a	6b	6c	7	8	9

交换法能保证稳定性吗？

- ❖ 若采用平移法，如此即可保证，重复元素在列表中的相对次序，与其插入次序一致



性能分析

❖ 共迭代 n 次，在第 k 次迭代中

- selectMax() 为 $\Theta(n - k)$

//算术级数

- swap() 为 $\mathcal{O}(1)$

//或 remove() + insert()

故总体复杂度应为 $\Theta(n^2)$

❖ 尽管如此，元素的**移动**操作远远少于起泡排序

//实际更为费时

也就是说， $\Theta(n^2)$ 主要来自于元素的**比较**操作

//成本相对更低

❖ 可否...每轮只做 $\mathcal{O}(n)$ 次比较，即找出当前的最大元素？

❖ 可以! ...利用高级数据结构，selectMax()可改进至 $\mathcal{O}(\log n)$

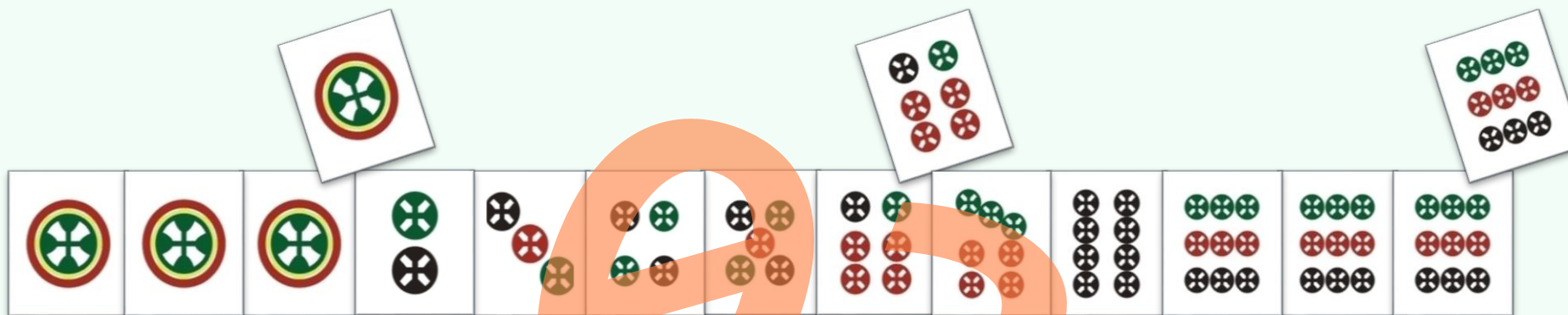
//稍后分解

当然，如此立即可以得到 $\mathcal{O}(n \log n)$ 的排序算法

//保持兴趣

列表

插入排序

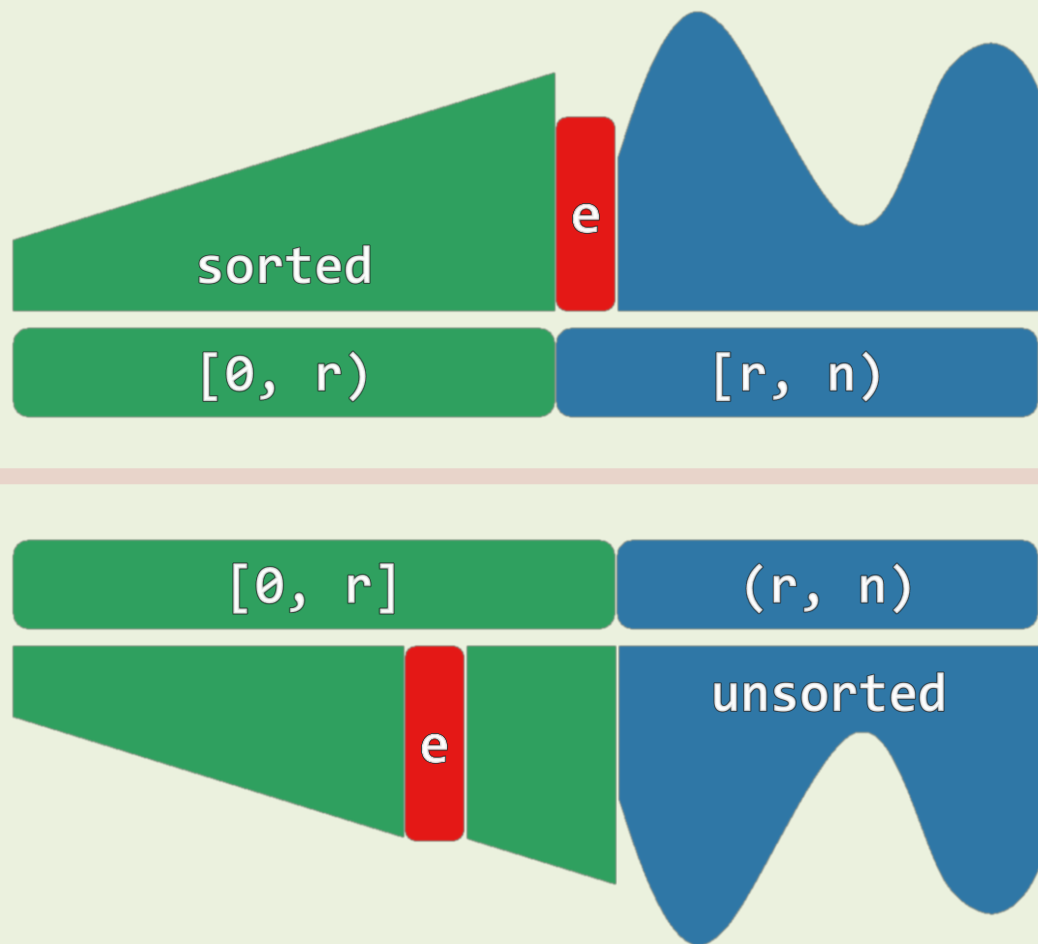


一语未了，只见宝玉笑嘻嘻的掇了一枝红梅进来，众丫鬟忙已接过，插入瓶内。

邓俊辉

deng@tsinghua.edu.cn

减而治之



❖ 始终将序列视作两部分：

- 前缀 $S[0, r)$ ：有序
- 后缀 $U[r, n)$ ：待排序

❖ 初始化： $|S| = r = 0$

❖ 反复地，针对 $e = A[r]$

- 在 S 中查找适当位置
- 插入 e
- $r++$

实例

			5	2	7	4	6	3	1
iteration	sorted prefix	e	random suffix						
$\emptyset$	^	5	2	7	4	6	3	1	
1	5	2	7	4	6	3	1		
2	2 5	7	4	6	3	1			
3	2 5 7	4	6	3	1				
4	2 4 5 7	6	3	1					
5	2 4 5 6 7	3	1						
6	2 3 4 5 6 7	1							
7	1 2 3 4 5 6 7								

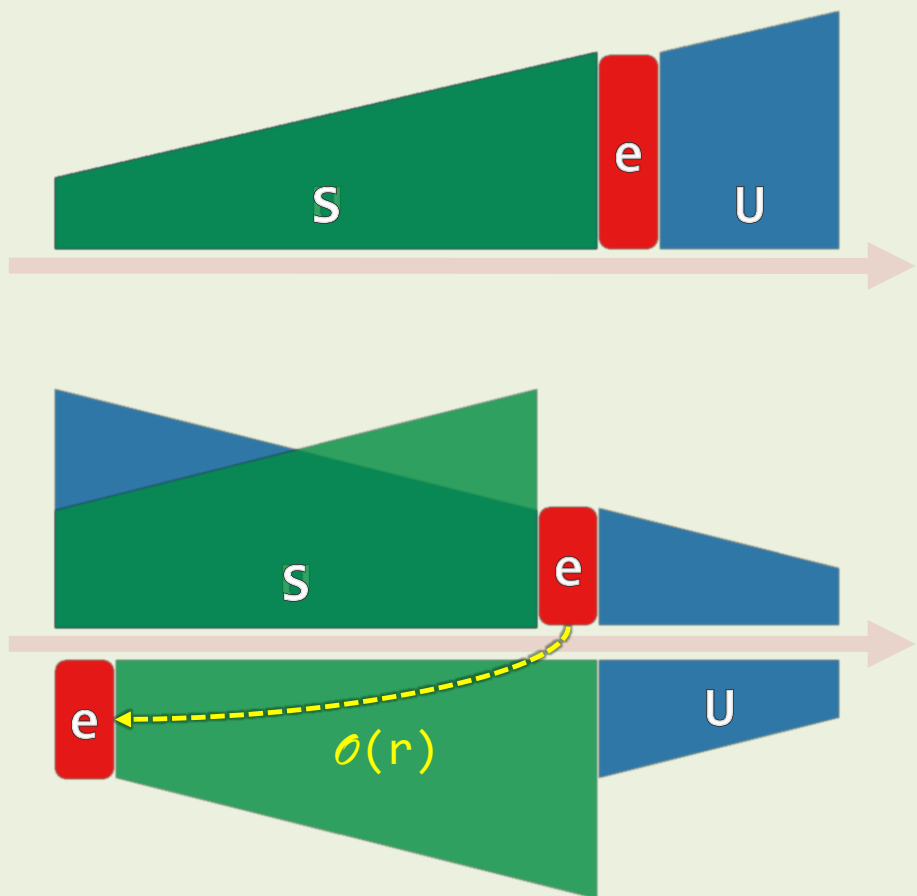
实现

```
template <typename T> void List<T>::insertionSort( ListNodePosi<T> p, Rank n ) {  
    for ( Rank r = 0; r < n; r++ ) { //逐一引入各节点, 由 $s_r$ 得到 $s_{r+1}$   
        insert( search( p->data, r, p ), p->data ); //查找 + 插入  
        p = p->succ; remove( p->pred ); //转向下一节点  
    } //n次迭代, 每次 $O(r + 1)$   
} //仅使用 $O(1)$ 辅助空间, 属于就地算法
```

无需移动的元素如何特判?

- ❖ 得益于此前约定的search()接口**语义**, 前缀的确总是**保持有序**, 而且**稳定**
- ❖ 验证以下情况, 体会**哨兵**的作用: p在前缀中有**雷同**; p在前缀中最**小**/最**大**; 前缀为**空**; ...

最好 & 最坏



❖ **最好:** $O(n)$ //比如, 已经有序, 相当于**验证**

❖ **最坏:** $O(n^2)$ //比如, 完全逆序

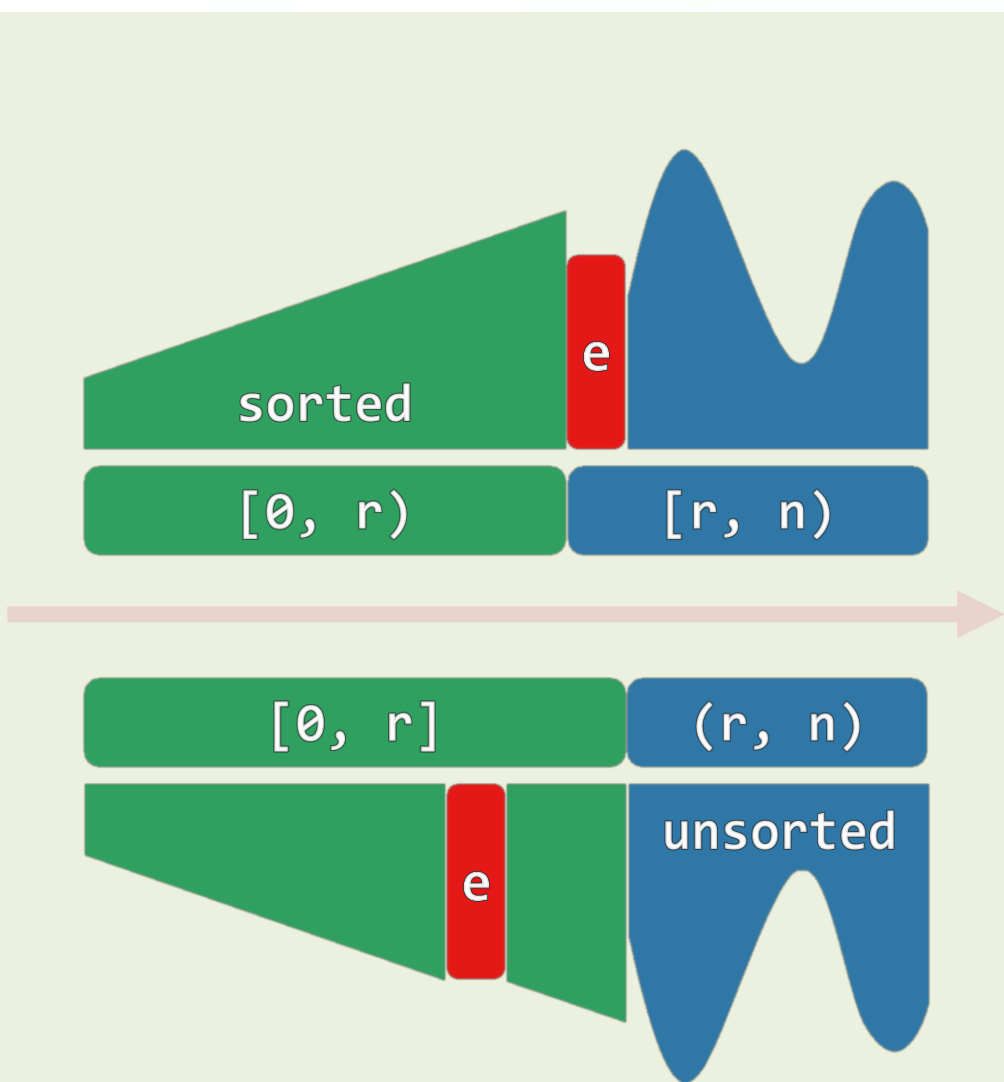
❖ **有实质的差异!**

Selectionsort呢?

❖ **主要消耗来自查找**

为何不...改用**binSearch()**?

平均 ~ 期望



❖ 若知道插入**每个元素**所需的**期望**时间
其**总和**即时总体的**期望**成本

❖ 后向分析：当前前缀 $[0, r]$ 中，谁是**最后**插入的？

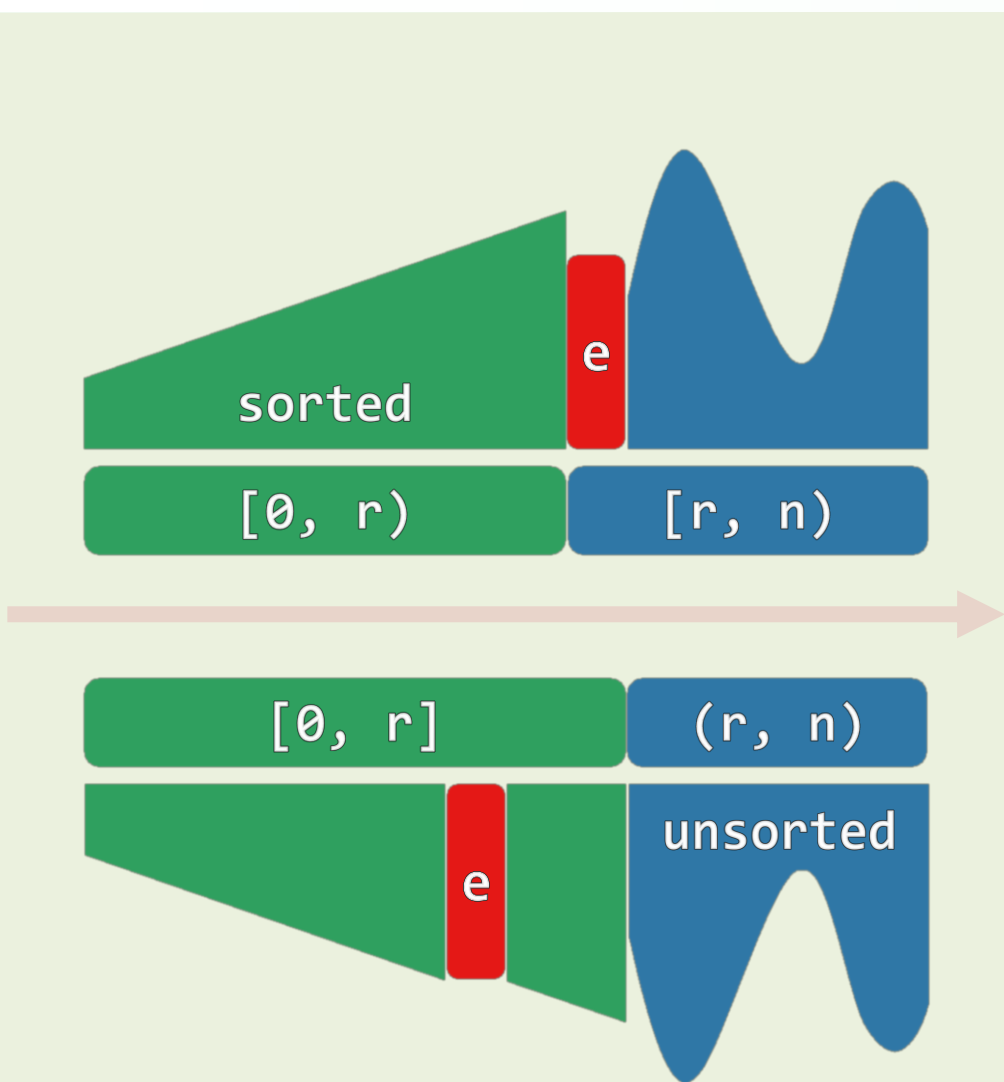
❖ 无非 $r+1$ 种情况，且概率均等

数学期望：
$$1 + \sum_{k=0}^r k / (r + 1) = \underline{1 + r/2}$$

❖ 故总和为：
$$\sum_{r=0}^{n-1} \underline{(1 + r/2)} = \mathcal{O}(n^2)$$

❖ 有的元素可能**无需**移动，特判并**节省**？

输入敏感



❖ 从 $\mathcal{O}(n)$ 到 $\mathcal{O}(n^2)$, 中间情况如何?

❖ 有序/乱序的程度

可简明度量, 而且

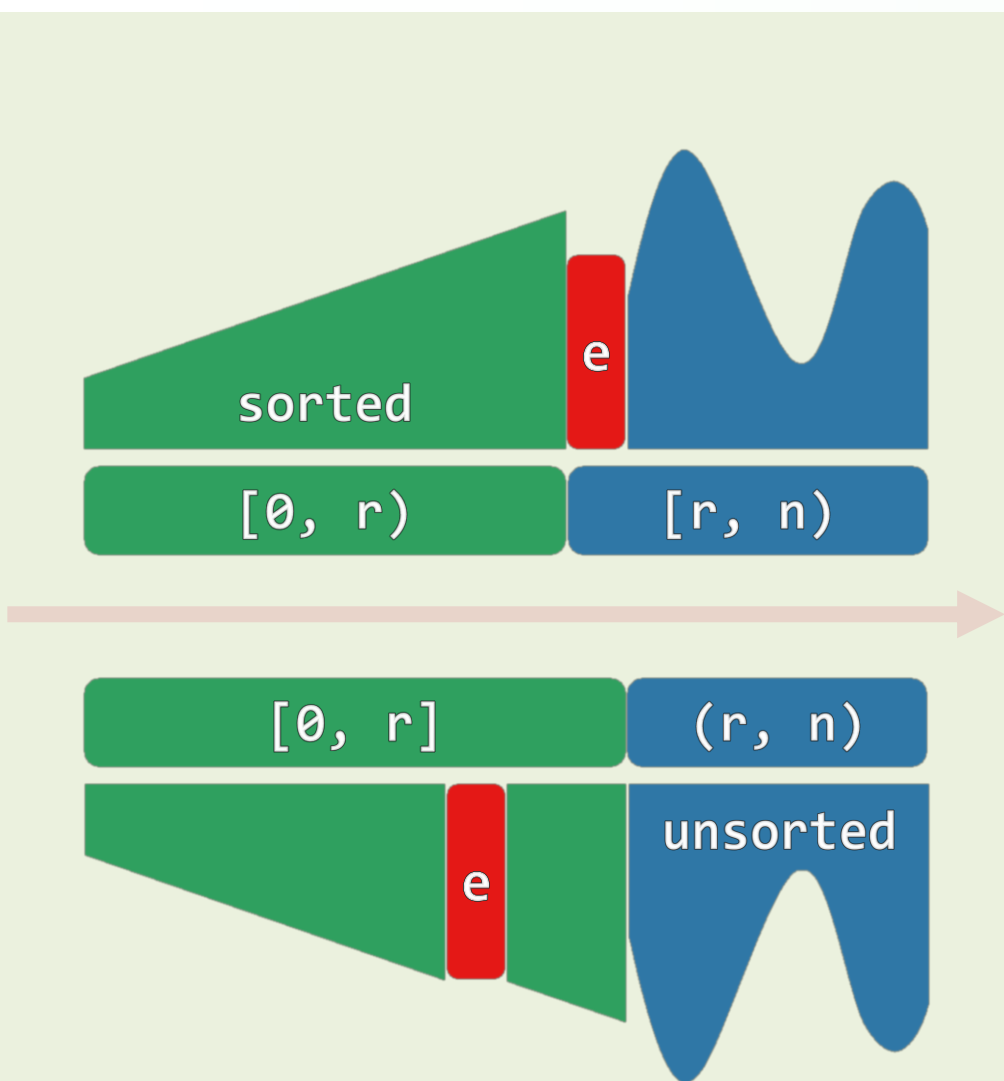
与时间成本之成正比...

❖ 输入敏感性/input-sensitivity

// 量出制入, 丰俭由人

(稍后详解)

在线



❖ 日常牌戏中，为何**你我他**都用它？

❖ 改用**其他**算法...

发牌完毕，才能各自开始**理牌**

❖ Insertionsort

可以**边发边理**；**发完即理好**

❖ online：在数据完全就绪之前，即可开始计算！

// 何必一味追求**结果**，不妨充分享受**过程**

Inversion

❖ 考查序列 $A[0, n)$, 设元素之间可比较大小

$\langle i, j \rangle$ is called an inversion if $0 \leq i < j < n$ and $A[i] > A[j]$

❖ 为便于统计, 可将逆序对统一记到**后者**的账上

$$\mathcal{I}(j) = \left\| \{ 0 \leq i < j \mid A[i] > A[j] \text{ and hence } \langle i, j \rangle \text{ is an inversion} \} \right\|$$

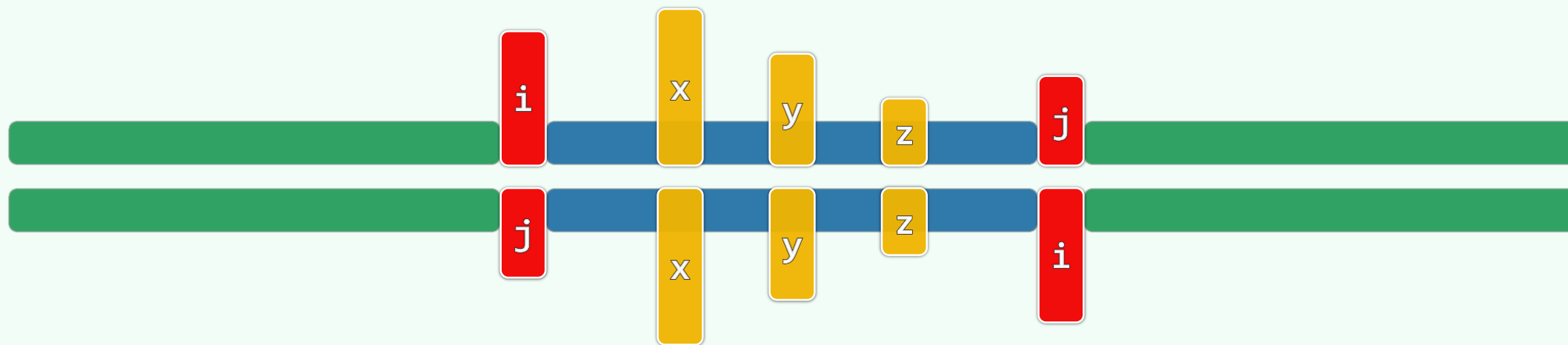
❖ 例: $A[] = \{ 5, 3, 1, 4, 2 \}$ 中, 共有 $0 + 1 + 2 + 1 + 3 = 7$ 个逆序对

$A[] = \{ 1, 2, 3, 4, 5 \}$ 中, 共有 $0 + 0 + 0 + 0 + 0 = 0$ 个逆序对

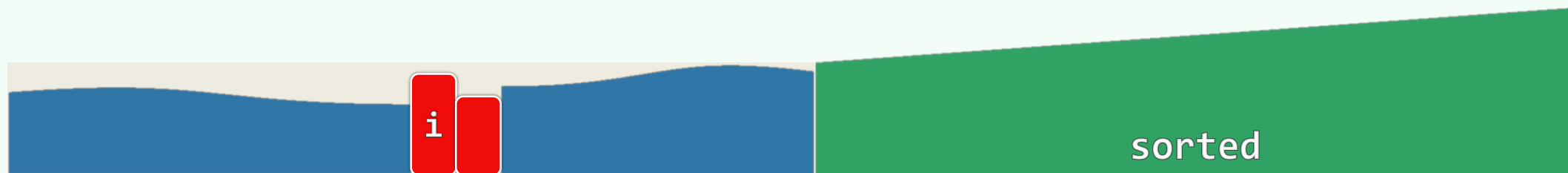
$A[] = \{ 5, 4, 3, 2, 1 \}$ 中, 共有 $0 + 1 + 2 + 3 + 4 = 10$ 个逆序对

❖ 显然, 逆序对总数 $\mathcal{I} = \sum_j \mathcal{I}(j) \leq \binom{n}{2} = \mathcal{O}(n^2)$

Bubblesort



❖ 在序列中交换一对逆序元素，逆序对总数必然减少



❖ 在序列中交换一对**紧邻的**逆序元素，逆序对总数恰好**减一**

BubbleSort是输入敏感的吗？

❖ 因此对于Bubblesort算法而言，**交换**操作的次数恰等于输入序列所含**逆序对**的总数

Insertionsort

e 的前缀经过插入顺序已经不同于原始, $I(r)$ 仍然保持不变吗?

❖ 针对 $e = A[r]$ 的那一步迭代

恰好需要做 $I(r)$ 次比较

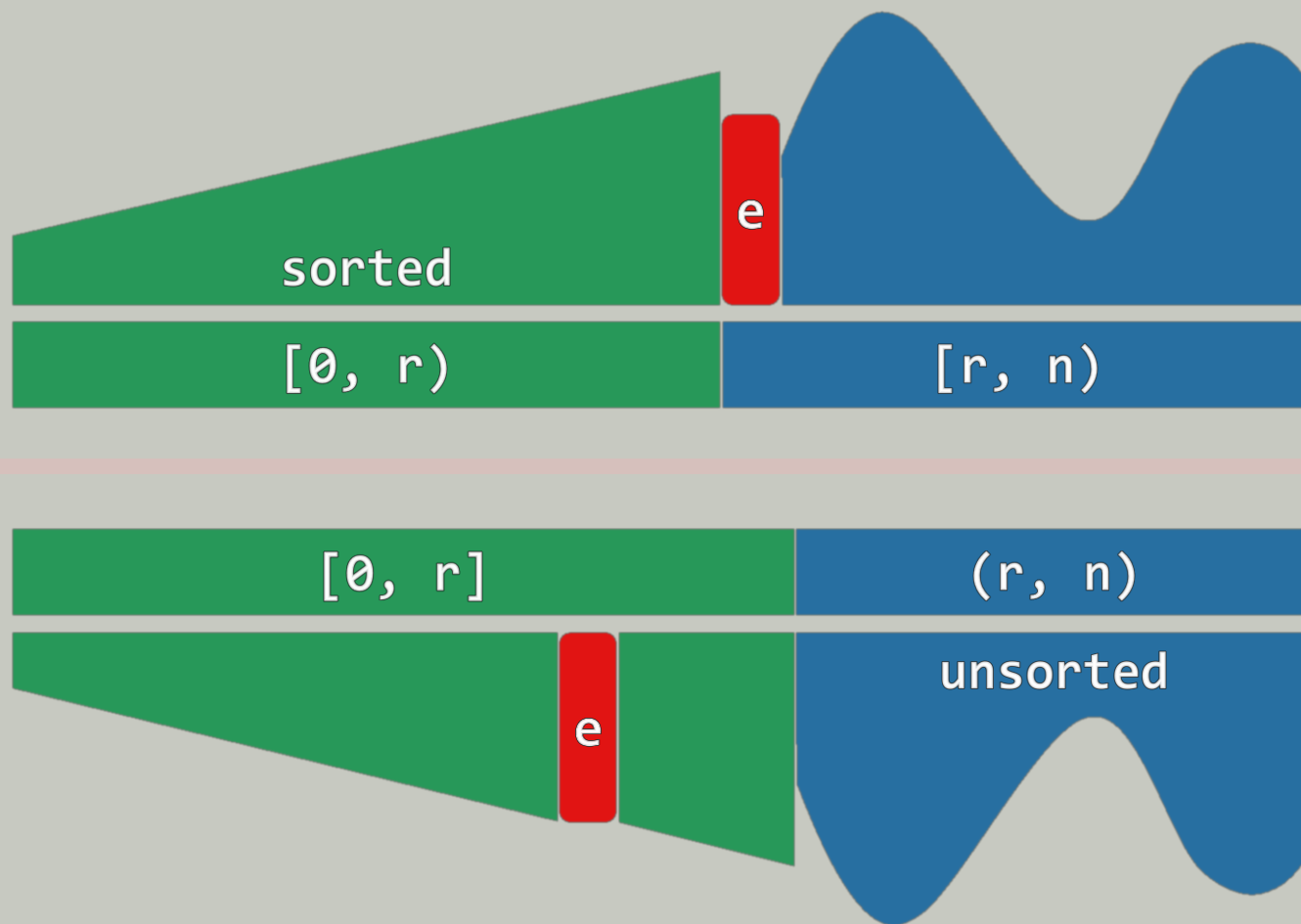
❖ 若共含 I 个逆序对, 则

- 关键码比较次数为 $O(I)$

- 运行时间为 $O(n + I)$

//习题[3-11]

//输入敏感性



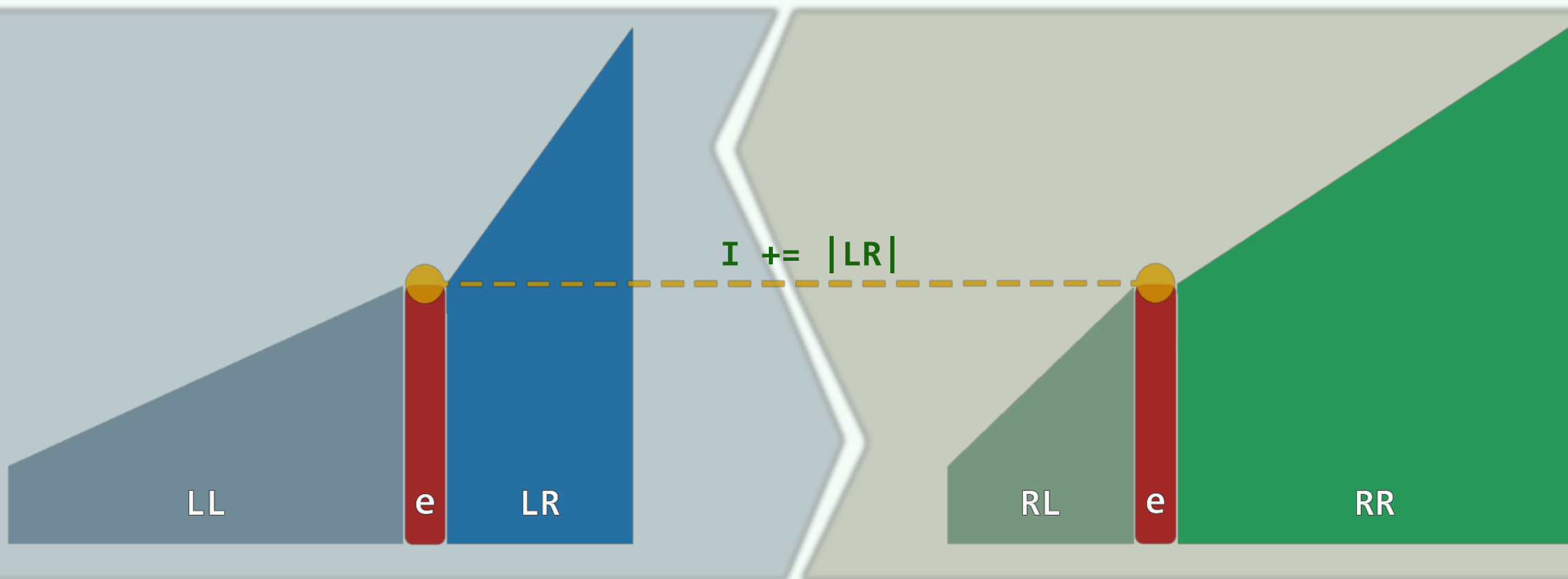
计数

❖ 任意给定一个序列

如何统计其中逆序对的总数?

❖ 蛮力算法需要 $\Omega(n^2)$ 时间

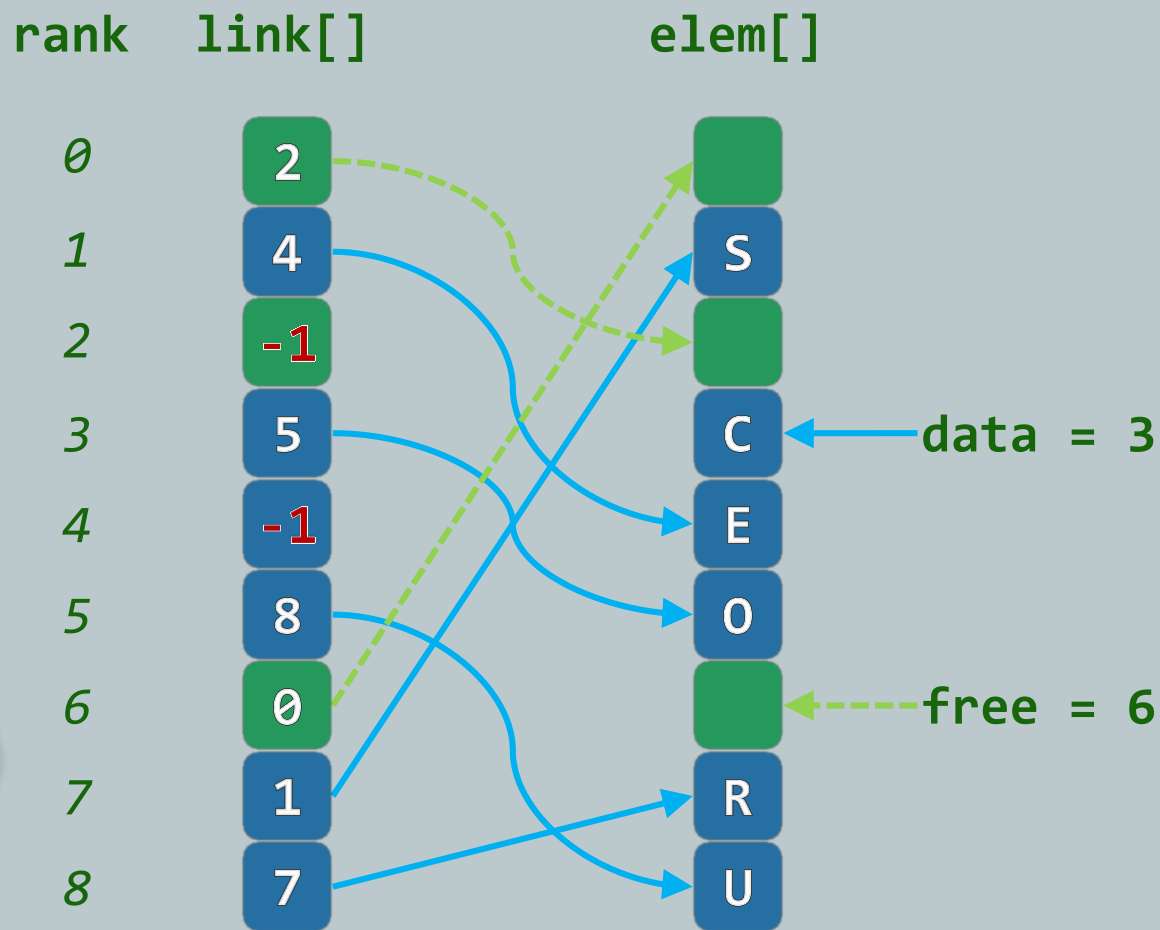
借助归并排序, 仅需 $\mathcal{O}(n \log n)$ 时间...



列表
游标

动机与构思

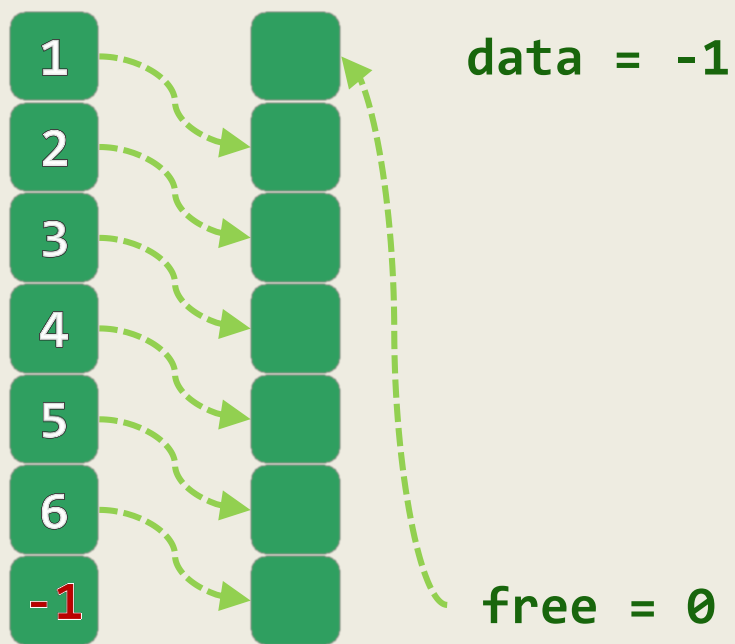
- ❖ 某些语言**不支持**指针类型，即便支持频繁的动态空间分配也**影响总体效率**
- ❖ 此时，又当如何实现列表结构呢？
- ❖ 利用线性数组，以游标方式模拟列表
 - elem[]: 对外可见的**数据项**
 - link[]: 数据项之间的**引用**
- ❖ 维护逻辑上**互补**的列表data和free
- ❖ 在插入或删除元素时，应如何调整？



实例 (1/4)

init(7)

link[] elem[]

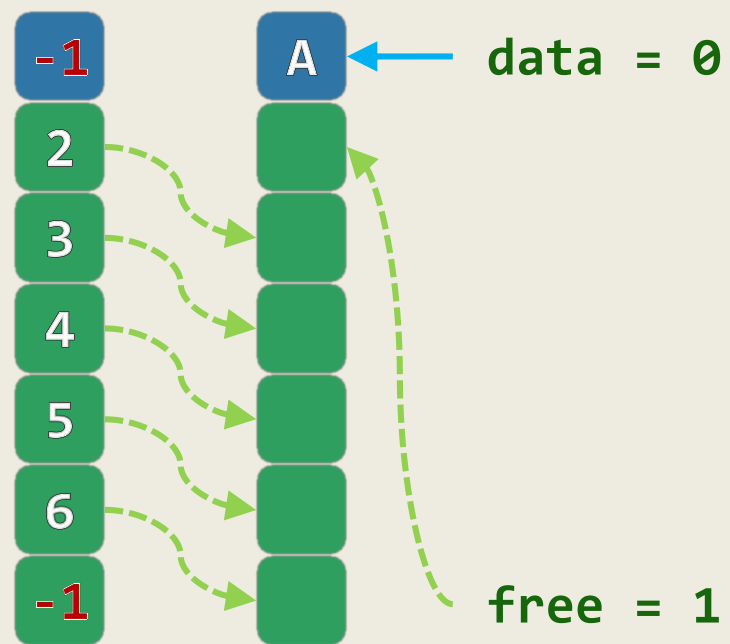


insert('A')

rank

link[] elem[]

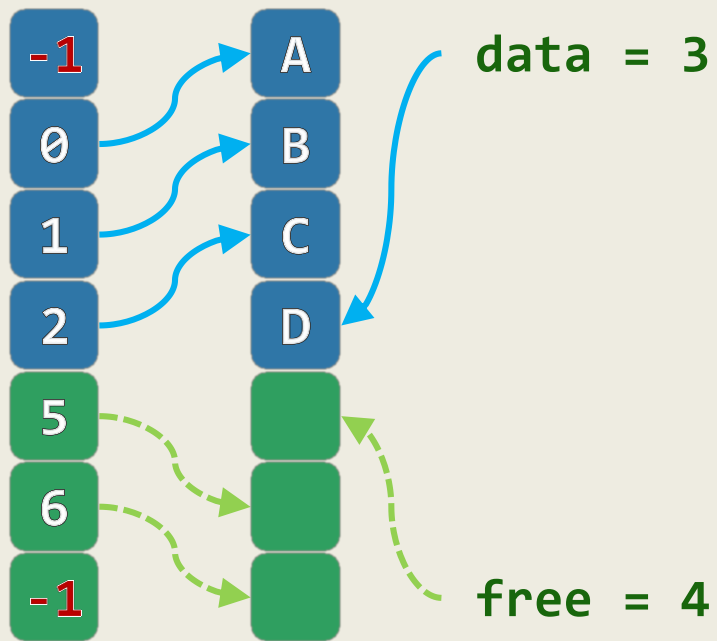
0
1
2
3
4
5
6



实例 (2/4)

insert('C')
insert('D')

link[] elem[]

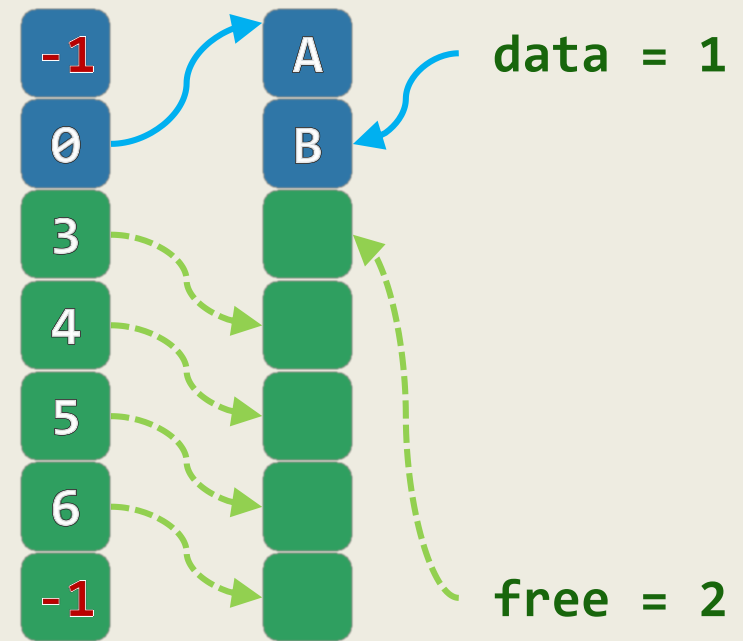


insert('B')

rank

link[] elem[]

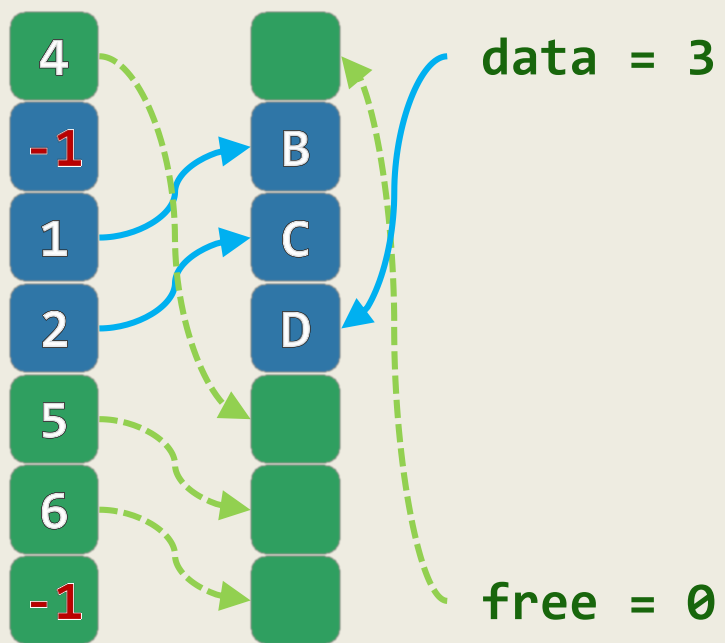
0
1
2
3
4
5
6



实例 (3/4)

remove('A')

link[] elem[]

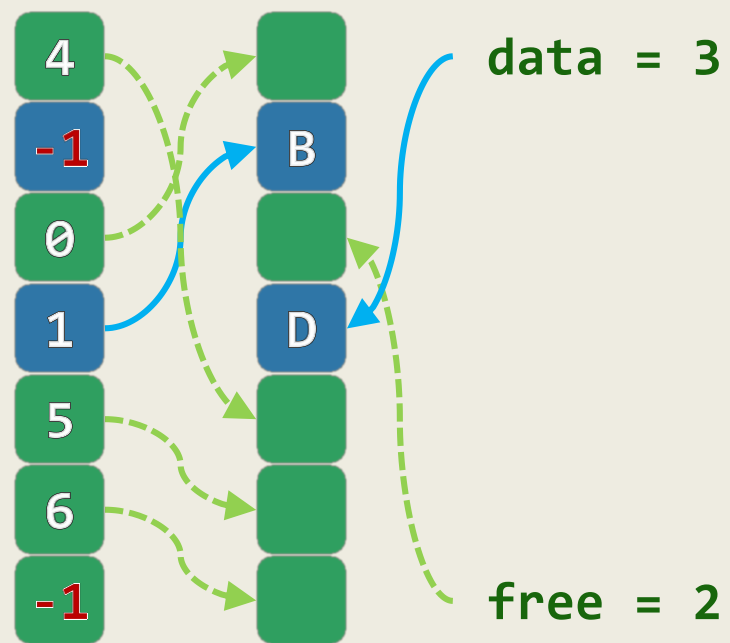


remove('C')

rank

link[] elem[]

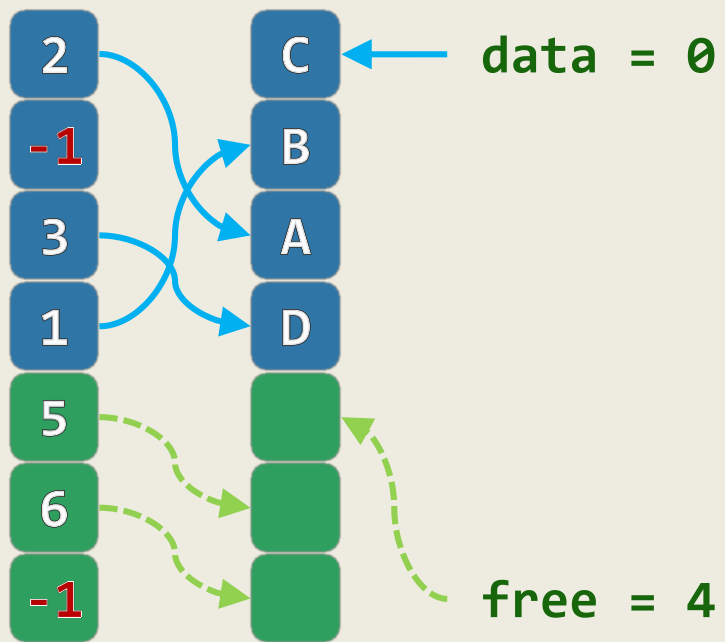
0
1
2
3
4
5
6



实例 (4/4)

insert('C')

link[] elem[]



insert('A')

rank

link[] elem[]

0
1
2
3
4
5
6

